

SELF-ATTENTION DOES NOT NEED $O(n^2)$ MEMORY

A PREPRINT

Markus N. Rabe and Charles Staats
Google Research
{mrabe,cstaats}@google.com

ABSTRACT

We present a very simple algorithm for attention that requires $O(1)$ memory with respect to sequence length and an extension to self-attention that requires $O(\log n)$ memory. This is in contrast with the frequently stated belief that self-attention requires $O(n^2)$ memory. While the time complexity is still $O(n^2)$, device memory rather than compute capability is often the limiting factor on modern accelerators. Thus, reducing the memory requirements of attention allows processing of longer sequences than might otherwise be feasible. We provide a practical implementation for accelerators that requires $O(\sqrt{n})$ memory, is numerically stable, and is within a few percent of the runtime of the standard implementation of attention. We also demonstrate how to differentiate the function while remaining memory-efficient. For sequence length 16384, the memory overhead of self-attention is reduced by 59X for inference and by 32X for differentiation.

1 Introduction

Attention (Bahdanau et al., 2015) is widely used in modern neural architectures. In particular, it is the heart of the Transformer architecture (Vaswani et al., 2017), which has revolutionized Natural Language Processing (Devlin et al., 2019), and found wide-spread adoption across several research areas since then.

Given a query $q \in \mathbb{R}^d$ and lists of keys and values $k_1, \dots, k_n$ and $v_1, \dots, v_n \in \mathbb{R}^d$ of length n , *attention* is defined as follows:

$$s_i = \text{dot}(q, k_i), \quad s'_i = \frac{e^{s_i}}{\sum_j e^{s_j}}, \quad \text{attention}(q, k, v) = \sum_i v_i s'_i.$$

The result of the attention operation for a single query, is hence a weighted sum of the value vectors, where the weights are the softmax of the dot products of the query and the keys.

The straight-forward implementation of the attention operation above requires us to first compute and remember s_i for all i , leading to a $O(n)$ time and memory complexity for each query. Transformers use *self-attention*, which issues a separate query for each position in the sequence, so the overall time and space complexity is $O(n^2)$.

In many works the quadratic time and space complexity of self-attention has been used as the motivation for the investigation of variants of the original attention mechanism and architectures with more favorable complexity classes (Kitaev et al., 2020; Roy et al., 2021; Zaheer et al., 2020; Choromanski et al., 2020; Wang et al., 2020; Ren et al., 2021; Child et al., 2019; Tay et al., 2021; Wang et al., 2020; Ma et al., 2021; Shen et al., 2021; Qiu et al., 2020). Modern accelerator hardware, such as GPUs and TPUs, are often memory constrained for applications in deep learning, while compute is relatively cheap. So the space complexity of transformers is a particular concern, c.f. Kitaev et al. (2020); Roy et al. (2021); Zaheer et al. (2020).

In this work, we present new algorithms for attention and self-attention that require only constant memory and logarithmic memory, respectively. The basic algorithm is very simple; but it requires a trick to make it numerically feasible (see Section 3). We also present an implementation in JAX (Bradbury et al., 2018), which runs efficiently on TPUs, and requires $O(\sqrt{n})$ memory for self-attention (see Section 4).

Unlike other works that aim to reduce the memory complexity of attention, the memory-efficient algorithm for attention that we suggest is not an approximation, but computes the same function. We can hence use the memory-efficient

algorithm as a drop-in replacement for other attention implementations to save memory. This may allow us to reconsider architecture choices, or scale to new datasets that require longer, dense attention. However, our algorithm still requires $O(n^2)$ time complexity for self-attention and $O(n)$ time complexity for single-query attention, and the various efficient, long-context attention mechanisms remain an interesting alternative to (dense) attention.

2 Algorithm

First, we present the algorithm for the attention operation with a single query and extend the algorithm to self-attention at the end of this Section. We observe that the division by $\sum_j e^{s_j}$ can be moved to the very end of the attention operation using the distributive law:

$$s_i = \text{dot}(q, k_i), \quad s'_i = e^{s_i}, \quad \text{attention}(q, k, v) = \frac{\sum_i v_i s'_i}{\sum_j s'_j}. \quad (1)$$

After publishing our initial draft, we were made aware that (1) is a rediscovery of the “lazy softmax” method of Jang et al. (2019, equation 4). Unfortunately their paper went in a different direction and did not discuss the memory complexity implications and other innovations we present in the remainder of this paper. For more details see Section 6.

This can be computed with constant memory: The memory overhead of this algorithm consists of a vector $v^* \in \mathbb{R}^d$ and a scalar $s^* \in \mathbb{R}$, both initialized with 0. Given the query q , keys $k_1, \dots, k_n$ and values $v_1, \dots, v_n$, we process the keys and values in sequence. Given a key value pair k_i, v_i , we compute $s_i = \text{dot}(q, k_i)$ and update $v^* \leftarrow v^* + v_i e^{s_i}$ and $s^* \leftarrow s^* + e^{s_i}$. After processing all keys and values, we divide $\frac{v^*}{s^*}$ to get the final result.

The analysis of space complexity assumes that inputs are given in a particular order: we first read the query, and then a list of *pairs* of keys and values. If the inputs are provided in a different order, we have to additionally store an index into the sequence, requiring $O(\log n)$ memory instead.

To extend this algorithm to *self*-attention, we compute the results to all queries sequentially. This requires just one additional index into the list of queries, giving rise to the $O(\log n)$ memory complexity. Note that the operation produces outputs that are linear in the size of the number of queries, i.e., $O(n)$, which is not counted towards the space complexity.

3 Numerical Stability

The formulation of standard attention that we presented in the Introduction, as well as our memory-efficient algorithm, are not numerically stable when using floating point arithmetic, because the softmax exponentiates the scores. For scores ≥ 89 the exponentiation results in `inf` (for `bfloat16` and `float32`), which will be carried through to the final result of the attention operation. In practice, the softmax is implemented by subtracting the maximum score from all scores. This does not change the result of the softmax, but avoids this numerical problem.

Our incremental computation of the sum of exponentiated scores (and the values times the scores) does not immediately allow for the same trick, as the maximum may depend on the last score in the sequence. But the subtraction cannot be delayed either, since the scores must be exponentiated before they can be added to the cumulative sum.

To resolve this problem, we introduce an additional scalar, which keeps track of the maximum score that the incremental algorithm has seen so far, and we renormalize the sums of exponentiated values as needed: We initialize the vector $v^* \in \mathbb{R}^d$ and scalar $s^* \in \mathbb{R}$ with 0, and m^* with $-\text{inf}$. As before, given a key value pair k_i, v_i , we compute $s_i = \text{dot}(q, k_i)$, but then the algorithm differs slightly from Section 2. We first compute $m_i = \max(m^*, s_i)$ and update $v^* \leftarrow v^* e^{m^* - m_i} + v_i e^{s_i - m_i}$ and $s^* \leftarrow s^* e^{m^* - m_i} + e^{s_i - m_i}$ and $m^* \leftarrow m_i$. After processing all keys and queries, we divide $\frac{v^*}{s^*}$ to get the final result.

4 An Implementation For TPUs

In this section, we provide a version of the algorithm above that exploits the massive parallelism of modern hardware, such as GPUs or TPUs. The naive algorithm above is not trivial to parallelize for a compiler, as the incremental sum introduces a dependency across all keys and values.

We present the entire implementation, including the support for multiple attention heads and memory-efficient differentiation in Figure 1. The implementation does not optimize strictly for memory efficiency, but instead aims to strike a balance between simplicity, computational efficiency, and memory requirements.

```

1  import functools, jax, math
2  from jax import numpy as jnp
3
4  def _query_chunk_attention(query, key, value, precision, key_chunk_size=4096):
5      """Multi-head dot product attention with a limited number of queries."""
6      num_kv, num_heads, k_features = key.shape
7      v_features = value.shape[-1]
8      key_chunk_size = min(key_chunk_size, num_kv)
9      query = query / jnp.sqrt(k_features)
10
11     @functools.partial(jax.checkpoint, prevent_cse=False)
12     def summarize_chunk(query, key, value):
13         attn_weights = jnp.einsum('qhd,khd->qhk', query, key, precision=precision)
14         max_score = jnp.max(attn_weights, axis=-1, keepdims=True)
15         max_score = jax.lax.stop_gradient(max_score)
16         exp_weights = jnp.exp(attn_weights - max_score)
17         exp_values = jnp.einsum('vhf,qhv->qhf', value, exp_weights, precision=precision)
18         return exp_values, exp_weights.sum(axis=-1),
19             max_score.reshape((query.shape[0], num_heads)))
20
21     def chunk_scanner(chunk_idx):
22         key_chunk = jax.lax.dynamic_slice(
23             key, (chunk_idx, 0, 0),
24             slice_sizes=(key_chunk_size, num_heads, k_features))
25         value_chunk = jax.lax.dynamic_slice(
26             value, (chunk_idx, 0, 0),
27             slice_sizes=(key_chunk_size, num_heads, v_features))
28         return summarize_chunk(query, key_chunk, value_chunk)
29
30     chunk_values, chunk_weights, chunk_max = jax.lax.map(
31         chunk_scanner, xs=jnp.arange(0, num_kv, key_chunk_size))
32
33     global_max = jnp.max(chunk_max, axis=0, keepdims=True)
34     max_diffs = jnp.exp(chunk_max - global_max)
35     chunk_values *= jnp.expand_dims(max_diffs, axis=-1)
36     chunk_weights *= max_diffs
37
38     all_values = chunk_values.sum(axis=0)
39     all_weights = jnp.expand_dims(chunk_weights, -1).sum(axis=0)
40     return all_values / all_weights
41
42 def attention(query, key, value, precision=jax.lax.Precision.HIGHEST,
43               query_chunk_size=1024):
44     """Memory-efficient multi-head dot product attention."""
45     num_q, num_heads, q_features = query.shape
46
47     def chunk_scanner(chunk_idx, _):
48         query_chunk = lax.dynamic_slice(
49             query, (chunk_idx, 0, 0),
50             slice_sizes=(min(query_chunk_size, num_q), num_heads, q_features))
51         return (chunk_idx + query_chunk_size,
52             _query_chunk_attention(query_chunk, key, value, precision=precision))
53
54     _, res = jax.lax.scan(
55         chunk_scanner, init=0, xs=None, length=math.ceil(num_q / query_chunk_size))
56     return res.reshape(num_q, num_heads, value.shape[-1])

```

Figure 1: Implementation of memory-efficient attention suited for TPUs.

Sequence length	$n = 2^8$	2^{10}	2^{12}	2^{14}	2^{16}	2^{18}	2^{20}
Size of inputs and outputs	160KB	640KB	2.5MB	10MB	40MB	160MB	640MB
Memory overhead of standard attention	270KB	4.0MB	64MB	1GB	OOM	OOM	OOM
Memory overhead of memory-eff. attn.	270KB	4.0MB	16MB	17MB	21MB	64MB	256MB
Compute time on TPUv3	0.06ms	0.11ms	0.7ms	11.3ms	177ms	2.82s	45.2s
Relative compute speed	$\pm 5\%$	$\pm 5\%$	$-8 \pm 2\%$	$-13 \pm 2\%$	-	-	-

Table 2: Memory and time requirements of self-attention during **inference**.

To exploit the parallelism available in modern hardware, we split the computation into chunks at the cost of some additional memory. In the outer loop (lines 54-55), we split the queries into chunks of constant size, resulting in a linear number of iterations. In each iteration of the outer loop, we call `_query_chunk_attention`, which itself processes the keys and values in chunks (lines 30-31). The chunks are processed sequentially and each chunk is summarized independently (lines 12 to 19). Assuming a chunk size of $\sqrt{n}$ for the keys and values, we hence obtain $\sqrt{n}$ summaries, giving rise to the $O(\sqrt{n})$ memory complexity.

After the summaries are computed, they need to be rescaled (lines 33 to 36) along the lines of Section 3, before we return the values divided by the weights (line 40). The result of each iteration of the outer loop is directly written to the output tensor `res` (line 54), so that no additional memory is consumed across iterations. (A multi-stage summarization approach could achieve $O(\log n)$ but would complicate the implementation.)

While a constant chunk size for the queries and a chunk size of $\sqrt{n}$ for the keys and values is optimal for memory consumption, the runtime is also affected by the choice of chunk size in practice, which is heavily affected by the choice of hardware. Ultimately, we have to leave this trade-off to the programmer, and expose the chunk sizes as arguments `query_chunk_size` and `key_chunk_size`. In Figure 1 we provide default values for the chunk sizes that lead to minimal runtime impact on TPU, while still providing significant memory savings.

5 Empirical Analysis

In this section, we experimentally compare the memory requirements and runtime performance of the suggested algorithm compared to the implementation of attention currently provided by Flax (Heek et al. (2020), see `flax/linen/attention.py`). We open-sourced the code of our implementation and most of the evaluation as a colab to help others reproduce the results: https://github.com/google-research/google-research/tree/master/memory_efficient_attention.

5.1 Inference

In Table 2 we compare the memory requirements and the compute time of the memory-efficient attention implementation and the Flax implementation of attention. The size of inputs and outputs includes the query, key, and value tensors of dtype `bfloat16`, and the output tensor of dtype `float32`. We measure the memory overhead as the TPUs peak memory in excess of the input and output tensors. All computations were done on a single TPUv3 chip. For this experiment, we only use one attention head.

Our memory-efficient implementation of attention removes the memory bottleneck of self-attention, scaling at least to a sequence length of 1M. At this sequence length the algorithm is multiplying over 1 trillion combinations of queries and keys. The time complexity is still quadratic.

The “relative compute speed” of the implementations was computed as the median over 100 runs—but the numbers still fluctuated across multiple runs of the evaluation and we only provide them to demonstrate that the runtime performance is roughly similar. Please note that this experiment analyzes the attention operation in isolation; the measured relative performance is not necessarily the same when the operations are embedded in larger architectures. In fact, we observed a slight increase in steps/sec of about 4% when training a small Transformer.

For all cases where the standard attention would not OOM (i.e. require $> 16\text{GB}$ device memory), we checked that the results of the two implementations are within 1.8×10^{-7} for inputs drawn from a normal distribution with standard deviation 1 (measured as the maximal absolute difference of any dimension in a self-attention over sequence length 2^{14}).

Sequence length	$n = 2^8$	2^{10}	2^{12}	2^{14}	2^{16}	2^{18}	2^{20}
Size of inputs and outputs	192KB	768KB	2.9MB	12MB	47MB	188MB	750MB
Memory overhead of standard attention	532KB	8.0MB	128MB	2.0GB	OOM	OOM	OOM
Memory overhead of memory-eff. attn.	532KB	8.0MB	41MB	64MB	257MB	1.0GB	4.0GB
Compute time on TPUv3	0.1ms	0.18ms	1.4ms	21ms	336ms	5.3s	85s
Relative compute speed	$\pm 5\%$	$\pm 5\%$	$-30 \pm 5\%$	$-35 \pm 5\%$	-	-	-

Table 3: Memory and time requirements of self-attention during **differentiation**. Note that the slowdown in compute speed is expected due to the use of checkpointing in memory-efficient attention.

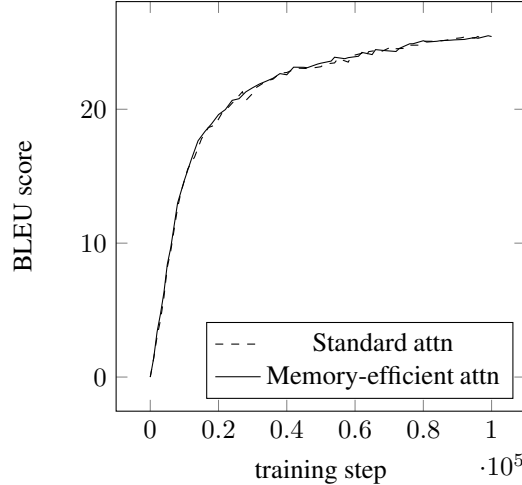


Figure 4: BLEU scores of a two Transformer models trained with standard attention and memory-efficient attention.

5.2 Differentiation

During the forward pass our algorithm saves memory by summarizing parts of the attention matrix sequentially, allowing it to forget the parts of the attention matrix it has summarized already. A naive application of differentiation would have to store all those intermediate results and our algorithm would lose its memory advantage entirely. So we apply checkpointing (Chen et al., 2016) in line 11 to the function that summarizes the individual chunks. The intermediate results can thus be forgotten during the forward pass and recomputed during backpropagation.

In Table 3 we compare runtime and peak memory during differentiation of our implementation to standard attention. We used the same setting as for the forward pass, but applied `jax.grad` to an arbitrarily chosen loss function (the sum of the results). The relative compute speed was reduced significantly compared to standard attention. This is expected when using checkpointing since some values must be recomputed during backpropagation.

Note that applying checkpointing to the standard attention algorithm would not achieve these results. The standard algorithm with checkpointing would forget the attention matrix after it is formed; our algorithm never forms the full attention matrix at all.

5.3 Training

We integrated our memory-efficient implementation into a simple Transformer architecture provided in the Flax library, and ran the WMT en-de translation experiment with the standard attention module and with the memory-efficient attention module. Throughout the training, the two implementations behaved almost identically. After 100K training steps, the evaluation accuracy reached 62.69 for the memory-efficient implementation and 62.59 for the standard implementation. This demonstrates that our memory-efficient implementation of self-attention can be used to replace existing implementations. Figure 4 illustrates that both models resulted in very similar BLEU scores. We used the default settings for the WMT en-de experiment as given in the Flax implementation, except that we had to deactivate example packing to simplify the masking code. This also required us to lower the learning rate to 0.005.

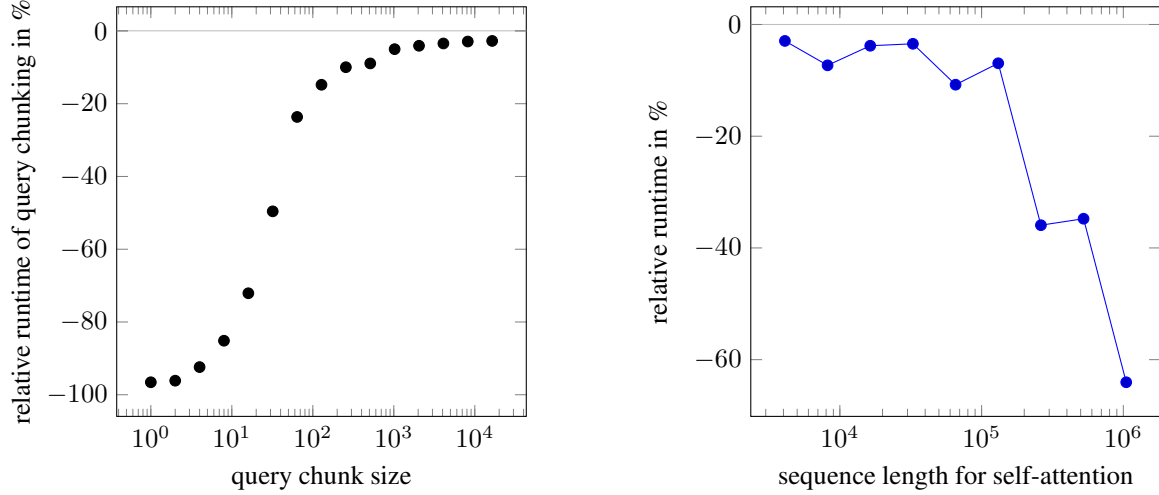


Figure 5: **Left:** Relative runtime of self-attention on sequence length 2^{15} using query chunking compared to standard attention. **Right:** Relative runtime of self-attention using query chunking compared to our memory-efficient algorithm, where both are restricted to the same amount of memory.

5.4 Comparison to Query Chunking

The algorithms introduced in this work chunk both the keys and the queries. Chunking the only queries has been explored already by Kitaev et al. (2020), but it is folklore that it slows down the computation significantly. In Figure 5 (left), we plot the runtime of self-attention using query-chunking for different query chunk sizes compared to dense self-attention: we see that for small chunk sizes (e.g. ≤ 64) the performance suffers indeed, but for large chunk sizes, the loss of performance is less significant. So, while lower memory consumption can be achieved by query chunking alone, small values for query chunking are impractical.

In comparison to query chunking, memory-efficient attention can save additional memory by chunking also the keys. This can help to keep the query chunk size at a desirable point given a fixed memory limit. In Figure 5, we constrained query chunking to the amount of memory that is used by memory-efficient attention with the default settings for key and query chunk size (see Table 2, “Memory overhead of memory-efficient att.”, we rounded the query chunk size towards the benefit of query chunking). We see that as the sequence length increases, query chunking eventually slows down significantly as the query chunk size has to be lowered to ≤ 64 , while memory-efficient attention does not suffer a major slowdown (see Table 2, “Relative compute speed”). So, in memory-constrained scenarios, memory-efficient attention can outperform query chunking.

6 Related Work

After publishing our initial draft, we were made aware that Jang et al. (2019) already observed that the division of the softmax operation can be delayed until the end of the attention operation (“lazy softmax”), similar to our Equation (1). But their paper does not discuss memory complexity at all. They also do not address numerical stability or backpropagation, and, as far as we know, there is no publicly available implementation of their work. Instead they use this algorithm to reduce the memory *bandwidth* for inference when sharding key-value pairs across multiple chips.

Dao et al. (2022) provide a CUDA implementation of memory-efficient attention and demonstrate that the reduced memory requirements can translate to significant speedups on GPUs. One reason why we do not observe the same performance gains in this paper is that standard self-attention already balances the available FLOPs and memory bandwidth of TPUs.

7 Conclusion

This paper presents a simple trick to reduce the memory requirement of (self-)attention dramatically, which appears to have been simply overlooked by the community. We hope that this short paper raises awareness of the fact that attention is not intrinsically memory-hungry, which may allow us to revisit some of the design choices in popular neural architectures and hardware architectures.

Acknowledgements

We want to thank Andrew Jaegle for discussions on this paper, and for experimenting with memory-efficient attention in the context of Perceiver (Hawthorne et al., 2022). We are glad to see that the algorithm proposed here has already found interest and would like to thank Rezaei (2021) and Wang (2022) for reimplementations in JAX and PyTorch with additional features like masking. We also want to thank DeLesley Hutchins for detailed feedback on our draft.

References

- Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. Neural machine translation by jointly learning to align and translate. In Yoshua Bengio and Yann LeCun (eds.), *ICLR*, 2015.
- James Bradbury, Roy Frostig, Peter Hawkins, Matthew James Johnson, Chris Leary, Dougal Maclaurin, George Necula, Adam Paszke, Jake VanderPlas, Skye Wanderman-Milne, and Qiao Zhang. JAX: composable transformations of Python+NumPy programs, 2018. URL <http://github.com/google/jax>.
- Tianqi Chen, Bing Xu, Chiyuan Zhang, and Carlos Guestrin. Training deep nets with sublinear memory cost. *CoRR*, abs/1604.06174, 2016.
- Rewon Child, Scott Gray, Alec Radford, and Ilya Sutskever. Generating long sequences with sparse transformers. *CoRR*, abs/1904.10509, 2019. URL <http://arxiv.org/abs/1904.10509>.
- Krzysztof Choromanski, Valerii Likhoshesterov, David Dohan, Xingyou Song, Andreea Gane, Tamás Sarlós, Peter Hawkins, Jared Davis, Afroz Mohiuddin, Lukasz Kaiser, David Belanger, Lucy J. Colwell, and Adrian Weller. Re-thinking attention with performers. *CoRR*, abs/2009.14794, 2020. URL <https://arxiv.org/abs/2009.14794>.
- Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. Flashattention: Fast and memory-efficient exact attention with io-awareness, 2022. URL <https://arxiv.org/abs/2205.14135>.
- Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: pre-training of deep bidirectional transformers for language understanding. In Jill Burstein, Christy Doran, and Thamar Solorio (eds.), *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL-HLT 2019, Minneapolis, MN, USA, June 2-7, 2019, Volume 1 (Long and Short Papers)*, pp. 4171–4186. Association for Computational Linguistics, 2019. doi: 10.18653/v1/n19-1423. URL <https://doi.org/10.18653/v1/n19-1423>.
- Curtis Hawthorne, Andrew Jaegle, Cătălina Cangea, Sebastian Borgeaud, Charlie Nash, Mateusz Malinowski, Sander Dieleman, Oriol Vinyals, Matthew Botvinick, Ian Simon, Hannah Sheahan, Neil Zeghidour, Jean-Baptiste Alayrac, João Carreira, and Jesse Engel. General-purpose, long-context autoregressive modeling with perceiver AR, 2022. URL <https://arxiv.org/abs/2202.07765>.
- Jonathan Heek, Anselm Levskaya, Avital Oliver, Marvin Ritter, Bertrand Rondepierre, Andreas Steiner, and Marc van Zee. Flax: A neural network library and ecosystem for JAX, 2020. URL <http://github.com/google/flax>.
- Hanhwi Jang, Joonsung Kim, Jae-Eon Jo, Jaewon Lee, and Jangwoo Kim. Mnnfast: A fast and scalable system architecture for memory-augmented neural networks. In *2019 ACM/IEEE 46th Annual International Symposium on Computer Architecture (ISCA)*, pp. 250–263, 2019.
- Nikita Kitaev, Łukasz Kaiser, and Anselm Levskaya. Reformer: The efficient transformer. *arXiv preprint arXiv:2001.04451*, 2020.
- Xuezhe Ma, Xiang Kong, Sinong Wang, Chunting Zhou, Jonathan May, Hao Ma, and Luke Zettlemoyer. Luna: Linear unified nested attention. *CoRR*, abs/2106.01540, 2021. URL <https://arxiv.org/abs/2106.01540>.
- Jiezhong Qiu, Hao Ma, Omer Levy, Wen-tau Yih, Sinong Wang, and Jie Tang. Blockwise self-attention for long document understanding. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pp. 2555–2565, 2020.
- Hongyu Ren, Hanjun Dai, Zihang Dai, Mengjiao Yang, Jure Leskovec, Dale Schuurmans, and Bo Dai. Combiner: Full attention transformer with sparse computation cost. *arXiv preprint arXiv:2107.05768*, 2021.
- Amin Rezaei. Memory efficient attention, 2021. URL <https://github.com/AminRezaei0x443/memory-efficient-attention>.
- Aurko Roy, Mohammad Saffar, Ashish Vaswani, and David Grangier. Efficient content-based sparse attention with routing transformers. *Trans. Assoc. Comput. Linguistics*, 9:53–68, 2021. URL <https://transacl.org/ojs/index.php/tacl/article/view/2405>.
- Zhuoran Shen, Mingyuan Zhang, Haiyu Zhao, Shuai Yi, and Hongsheng Li. Efficient attention: Attention with linear complexities. In *IEEE Winter Conference on Applications of Computer Vision, WACV 2021, Waikoloa*,

- HI, USA, January 3-8, 2021, pp. 3530–3538. IEEE, 2021. doi: 10.1109/WACV48630.2021.00357. URL <https://doi.org/10.1109/WACV48630.2021.00357>.
- Yi Tay, Mostafa Dehghani, Samira Abnar, Yikang Shen, Dara Bahri, Philip Pham, Jinfeng Rao, Liu Yang, Sebastian Ruder, and Donald Metzler. Long range arena : A benchmark for efficient transformers. In *9th International Conference on Learning Representations, ICLR 2021, Virtual Event, Austria, May 3-7, 2021*. OpenReview.net, 2021. URL <https://openreview.net/forum?id=qVyeW-grC2k>.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in neural information processing systems*, pp. 5998–6008, 2017.
- Phil Wang. Memory efficient attention Pytorch, 2022. URL <https://github.com/lucidrains/memory-efficient-attention-p>.
- Sinong Wang, Belinda Z Li, Madian Khabsa, Han Fang, and Hao Ma. Linformer: Self-attention with linear complexity. *arXiv preprint arXiv:2006.04768*, 2020.
- Manzil Zaheer, Guru Guruganesh, Kumar Avinava Dubey, Joshua Ainslie, Chris Alberti, Santiago Ontañón, Philip Pham, Anirudh Ravula, Qifan Wang, Li Yang, and Amr Ahmed. Big bird: Transformers for longer sequences. In Hugo Larochelle, Marc’Aurelio Ranzato, Raia Hadsell, Maria-Florina Balcan, and Hsuan-Tien Lin (eds.), *Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020, NeurIPS 2020, December 6-12, 2020, virtual*, 2020. URL <https://proceedings.neurips.cc/paper/2020/hash/c8512d142a2d849725f31a9a7a361ab9-Abstract.html>.